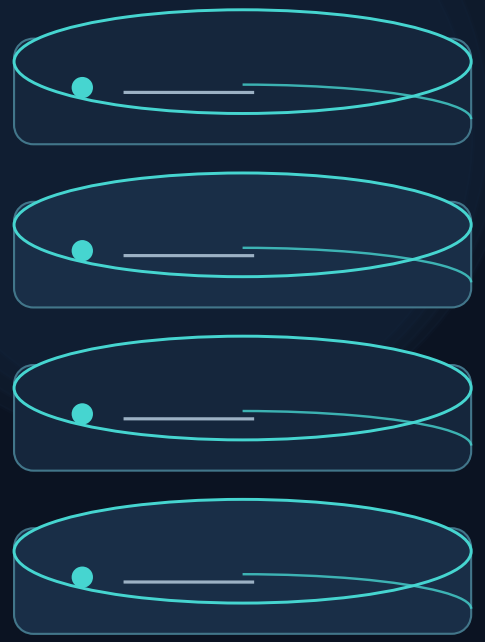
TECNOLOGIA NA PRÁTICA / MÓDULOS 4.3 A 4.6

Bancos de Dados Relacionais e SQL

Da modelagem às consultas que resolvem problemas reais.



MODELAR • CONSULTAR

PROFESSOR

Luis Felipe de Araujo

Nossa rota de hoje

Uma jornada: entender, praticar, testar e explicar.

01	**Modelagem**	Organizar antes de codificar	25 min
02	**CRUD**	Criar, consultar, atualizar e excluir	30 min
03	**JOIN**	Conectar tabelas e informações	25 min
04	**Agregações**	Transformar dados em indicadores	25 min
05	**Desafio final**	Construir um relatório real	15 min

Loja Byte: nosso laboratório

Um e-commerce fictício que usaremos do começo ao fim da aula.

CLIENTES

Quem compra

Ana, Bruno, Carla, Diego

PRODUTOS

O que vendemos

Teclado, Mouse, Monitor, Webcam

PEDIDOS

Cada compra realizada

Número, cliente e data

ITENS_PEDIDO

O conteúdo da compra

Produto, quantidade e preço

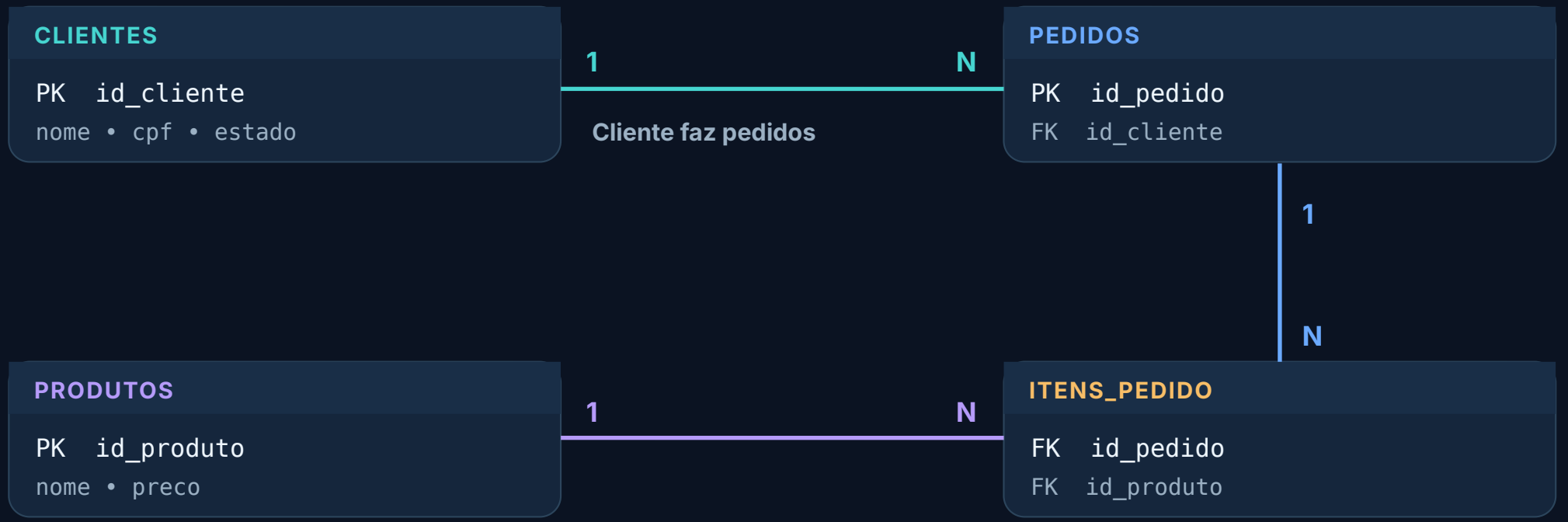
ANALOGIA DO DIA

Apps de delivery também conectam pessoas, produtos e pedidos sem repetir tudo em uma única tabela.

MER: desenhar antes de programar

O modelo mostra entidades e relacionamentos antes da criação de tabelas.

DIAGRAMA SIMPLIFICADO DA LOJA BYTE



Entidades: os personagens do sistema

Entidades são os objetos ou conceitos sobre os quais precisamos armazenar dados.

Cliente

Quem utiliza a loja

Produto

O que está à venda

Pedido

A compra efetuada

ItemPedido

O detalhe de cada compra

PERGUNTA PARA A TURMA

Qual opção é uma entidade?

A) preco

B) Restaurante

C) data_entrega

Atributos: as características

Um atributo precisa de nome, tipo de dado e, quando necessário, restrições.

COLUNA	TIPO	PARA QUE SERVE
id_cliente	INT	Identificador único (PK)
nome	VARCHAR(80)	Texto de tamanho variável
cpf	CHAR(11)	Código com zeros à esquerda
estado	CHAR(2)	Sigla da UF
preco	DECIMAL(10,2)	Valor monetário

MYSQL / SQL

```sql
CREATE TABLE clientes (
  id_cliente INT PRIMARY KEY,
  nome VARCHAR(80) NOT NULL,
  cpf CHAR(11) UNIQUE,
  estado CHAR(2) NOT NULL
);
```

PEGADINHA

Por que CPF não deve ser INT? Porque é um identificador, não uma quantidade; um tipo textual preserva zeros à esquerda.

Relacionamentos: como se conectam?

Cardinalidade descreve quantas ocorrências de cada entidade podem estar associadas.

1 : 1

Um para um

Conta ↔ Perfil

Uma conta ↔ um perfil

1 : N

Um para muitos

Cliente ↔ Pedidos

Um cliente → vários pedidos

N : N

Muitos para muitos

Pedidos ↔ Produtos

Intermediária: itens_pedido

ANALOGIA

Uma playlist tem muitas músicas; uma música pode estar em muitas playlists. É um relacionamento N:N.

PK e FK: a ponte entre as tabelas

A chave primária identifica; a chave estrangeira estabelece a referência.

CLIENTES

`id_cliente → PRIMARY KEY`

1:N

PEDIDOS

`id_cliente → FOREIGN KEY`

MYSQL / SQL

```sql
CREATE TABLE pedidos (
  id_pedido INT PRIMARY KEY,
  id_cliente INT NOT NULL,
  data_pedido DATE NOT NULL,
  FOREIGN KEY (id_cliente)
    REFERENCES clientes(id_cliente)
);
```

PERGUNTA

O que acontece se um pedido apontar para um cliente que não existe? A FK impede a referência inválida, quando a restrição está ativa.

Quiz: o que você já sabe?

Vote antes de discutir. Não vale consultar o slide anterior.

PERGUNTA PARA A TURMA

Qual campo seria a melhor PK de pedidos?

- **A** nome_cliente
- **B** id_pedido
- **C** valor_total

PERGUNTA PARA A TURMA

Como modelar Pedidos ↔ Produtos (N:N)?

- **A** Tabela intermediária
- **B** Repetir colunas de produto
- **C** Lista em um campo de texto

CRUD: quatro operações, quase todo app

Uma linguagem simples para pensar no ciclo de vida dos registros.

C	R	U	D
CREATE	**READ**	**UPDATE**	**DELETE**
`INSERT INTO`	`SELECT`	`UPDATE`	`DELETE FROM`
Adicionar	Consultar	Alterar	Remover

ANALOGIA

Um perfil de rede social: criar conta, visualizar, atualizar bio e excluir a conta.

Preparando nosso laboratório

Estes registros fictícios serão usados durante toda a prática.

CLIENTES

ID	NOME	UF
1	Ana	BA
2	Bruno	SP
3	Carla	BA
4	Diego	RJ

PRODUTOS

ID	PRODUTO	PREÇO
10	Teclado	R$ 120
20	Mouse	R$ 80
30	Monitor	R$ 950
40	Webcam	R$ 200

MYSQL / SQL

```sql
USE loja_byte;
SHOW TABLES;
SELECT * FROM clientes;
```

CHECKPOINT

Execute o script fornecido e verifique que existem quatro tabelas: clientes, produtos, pedidos e itens_pedido.

CREATE: inserindo um cliente

INSERT INTO cria uma nova linha na tabela.

MYSQL / SQL

```sql
START TRANSACTION;

INSERT INTO clientes
  (id_cliente, nome, estado)
VALUES (5, 'Eva', 'PE');

SELECT * FROM clientes
WHERE id_cliente = 5;

ROLLBACK;
```

DEPOIS DO INSERT

Eva · PE

ID 5 aparece no SELECT

MISSÃO DE 60 SEGUNDOS

Cadastre Rafael, do estado MG, com ID 6. Teste e depois reverta usando ROLLBACK.

READ: consultando com SELECT

Escolha colunas, filtre registros e defina a ordem de exibição.

MYSQL / SQL

```
01 SELECT nome, estado
02 FROM clientes
03 WHERE estado = 'BA'
04 ORDER BY nome;
```

RESULTADO

NOME	ESTADO
Ana	BA
Carla	BA

DESAFIO

Liste os produtos com preço acima de R$ 100: use SELECT, FROM e WHERE preco > 100.

UPDATE: alterando o registro certo

WHERE é a sua proteção contra alterações em massa.

MYSQL / SQL

```sql
START TRANSACTION;

UPDATE produtos
SET preco = 90.00
WHERE id_produto = 20;

SELECT nome, preco FROM produtos
WHERE id_produto = 20;
ROLLBACK;
```

CUIDADO: SEM WHERE

UPDATE produtos SET preco = 90.00;

É SQL válido, mas alteraria TODOS os produtos. O reajuste do Mouse virou promoção para a loja inteira.

BOA PRÁTICA

Faça um SELECT com o mesmo WHERE antes de atualizar.

DELETE: removendo sem provocar caos

DELETE FROM exclui linhas; WHERE define exatamente quais.

MYSQL / SQL

```sql
START TRANSACTION;

DELETE FROM clientes
WHERE id_cliente = 4;

SELECT * FROM clientes
WHERE id_cliente = 4;

ROLLBACK;
```

PERIGO REAL

DELETE FROM clientes;

Sem WHERE, o comando tenta excluir todas as linhas. Restrições de FK podem bloquear a operação.

REGRA DE OURO

Verifique os IDs, revise os vínculos e teste em transação. ROLLBACK só ajuda antes do COMMIT.

Pegadinha: um comando perigoso

O SQL abaixo roda. Mas o objetivo era alterar apenas o cliente 2.

MYSQL / SQL

```sql
UPDATE clientes
SET estado = 'SP';
```

PERGUNTA PARA A TURMA

O que falta para atualizar uma única pessoa?

A) ORDER BY id_cliente

B) WHERE id_cliente = 2

C) GROUP BY estado

JOIN: conectando as peças

Uma consulta pode combinar linhas de tabelas diferentes usando chaves relacionadas.

CLIENTES

id_cliente	nome
1	Ana
2	Bruno
3	Carla

FK

PEDIDOS

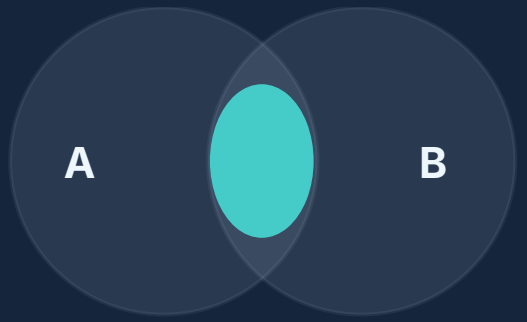
id_pedido	id_cliente
101	1
102	2
103	1

PERGUNTA

Se pedidos guarda apenas id_cliente, como encontrar o nome da pessoa que fez o pedido 101?

INNER, LEFT e RIGHT JOIN

Quem entra no resultado depende do tipo de junção.



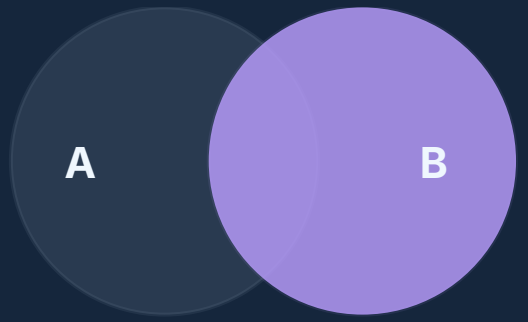
INNER JOIN

Só linhas com correspondência.



LEFT JOIN

Todas as linhas da esquerda.



RIGHT JOIN

Todas as linhas da direita.

Os diagramas ajudam a visualizar a ideia; SQL combina LINHAS, que podem se repetir em relações 1:N.

INNER JOIN: quem comprou?

Mostra apenas pedidos que encontram clientes correspondentes.

MYSQL / SQL

```sql
SELECT c.nome, p.id_pedido
FROM clientes c
INNER JOIN pedidos p
  ON c.id_cliente = p.id_cliente
ORDER BY p.id_pedido;
```

RESULTADO

CLIENTE	PEDIDO
Ana	101
Bruno	102
Ana	103
Carla	104

DESAFIO

Por que Ana aparece duas vezes? E por que Diego não aparece?

LEFT JOIN: quem ainda não comprou?

Preserve todos os clientes e identifique quem não tem pedidos.

MYSQL / SQL

```sql
SELECT c.nome, p.id_pedido
FROM clientes c
LEFT JOIN pedidos p
  ON c.id_cliente = p.id_cliente
WHERE p.id_pedido IS NULL;
```

RESULTADO

CLIENTE	PEDIDO
Diego	NULL

CASO DE NEGÓCIO

Marketing quer oferecer cupom a clientes cadastrados que nunca compraram. Este é um caso típico de LEFT JOIN + IS NULL.

RIGHT JOIN: olhando pelo outro lado

Preserva a tabela da direita, mesmo quando a esquerda não possui correspondência.

MYSQL / SQL

```sql
SELECT c.nome, p.id_pedido
FROM pedidos p
RIGHT JOIN clientes c
  ON p.id_cliente = c.id_cliente;
```

MESMO EFEITO DE:

```sql
clientes c
LEFT JOIN pedidos p
ON c.id_cliente = p.id_cliente
```

PERGUNTA

Se podemos trocar a ordem das tabelas e usar LEFT JOIN, por que também existe RIGHT JOIN?

Pegadinha: o SQL executa...

Mas o relacionamento compara as colunas erradas.

MYSQL / SQL

```sql
SELECT c.nome, p.id_pedido
FROM clientes c
INNER JOIN pedidos p
  ON c.id_cliente = p.id_pedido;
```

DISCUTA EM DUPLAS

Qual coluna deveria estar no ON? O resultado será vazio, duplicado ou correto?

De dados para decisões

Agora o SQL responde perguntas de negócio — não apenas lista registros.

COUNT

4 pedidos

Quantos registros?

SUM

R$ 1.510

Quanto faturamos?

AVG

R$ 337,50

Preço médio dos produtos

MIN / MAX

80 / 950

Menor e maior preço

PERGUNTA

O dono da Loja Byte quer saber qual estado mais compra. Precisamos de JOIN + agregação + GROUP BY.

COUNT e SUM: quantos e quanto?

COUNT conta registros; SUM acumula valores numéricos.

SQL / COUNT

```sql
SELECT COUNT(*) AS total_pedidos
FROM pedidos;
```

QUANTIDADE

4 pedidos

SQL / SUM

```sql
SELECT SUM(quantidade * preco_unitario)
  AS faturamento_total
FROM itens_pedido;
```

FATURAMENTO

R$ 1.510

EXTRA: COUNT(DISTINCT id_cliente) na tabela pedidos retorna 3 compradores.

AVG, MIN e MAX: lendo os extremos

Uma consulta para descobrir média, menor e maior preço.

MYSQL / SQL

```sql
SELECT
  AVG(preco) AS preco_medio,
  MIN(preco) AS menor_preco,
  MAX(preco) AS maior_preco
FROM produtos;
```

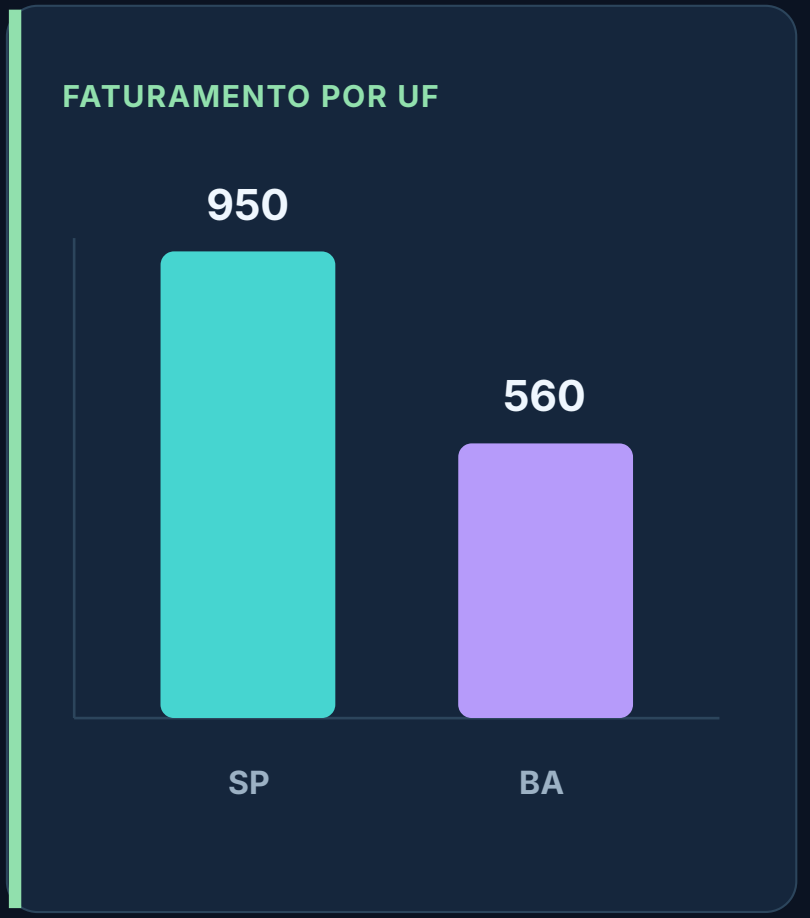
PREÇO MÉDIO	MENOR PREÇO	MAIOR PREÇO
R$ 337,50	R$ 80,00	R$ 950,00

GROUP BY: vendas por estado

Faça o SQL agrupar os registros e somar o faturamento de cada UF.

MYSQL / SQL

```sql
SELECT c.estado,
  SUM(i.quantidade * i.preco_unitario)
    AS faturamento
FROM clientes c
JOIN pedidos p
  ON p.id_cliente = c.id_cliente
JOIN itens_pedido i
  ON i.id_pedido = p.id_pedido
GROUP BY c.estado
ORDER BY faturamento DESC;
```



WHERE x HAVING: filtre no momento certo

WHERE filtra linhas antes do agrupamento; HAVING filtra grupos depois.

01 WHERE

Filtra pedidos por data

→

02 GROUP BY

Agrupa as linhas por estado

→

03 HAVING

Mantém grupos acima de R$ 500

MYSQL / SQL

```sql
SELECT c.estado,
  SUM(i.quantidade * i.preco_unitario) AS total
FROM clientes c
JOIN pedidos p ON p.id_cliente = c.id_cliente
JOIN itens_pedido i ON i.id_pedido = p.id_pedido
WHERE p.data_pedido >= '2026-11-01'
GROUP BY c.estado
HAVING SUM(i.quantidade * i.preco_unitario) > 500;
```

Pegadinha: WHERE ou HAVING?

Encontre o detalhe que impede a consulta de executar corretamente.

MYSQL / SQL

```sql
SELECT c.estado, COUNT(*) AS total
FROM clientes c
JOIN pedidos p ON p.id_cliente = c.id_cliente
WHERE COUNT(*) > 1
GROUP BY c.estado;
```

PERGUNTA PARA A TURMA

Como corrigir a agregação?

A) Trocar COUNT por SUM

B) Usar HAVING depois do GROUP BY

C) Remover GROUP BY

Desafio: quanto cada cliente gastou?

O proprietário quer um ranking completo — incluindo quem ainda não comprou.

REQUISITOS DA MISSÃO

- Nome do cliente e total gasto
- JOIN + GROUP BY
- Incluir quem nunca comprou
- Exibir R$ 0 quando não há compras
- Ordenar pelo maior gasto

MYSQL / SQL

```sql
SELECT
  c.nome,
  -- calcular o total gasto
FROM clientes c
-- conectar pedidos
-- conectar itens_pedido
-- agrupar por cliente
-- ordenar pelo total gasto
```

Gabarito: o relatório está pronto

LEFT JOIN + SUM + COALESCE + GROUP BY resolvem a missão.

MYSQL / SQL

```sql
SELECT c.nome,
  COALESCE(
    SUM(i.quantidade * i.preco_unitario), 0
  ) AS total_gasto
FROM clientes c
LEFT JOIN pedidos p
  ON p.id_cliente = c.id_cliente
LEFT JOIN itens_pedido i
  ON i.id_pedido = p.id_pedido
GROUP BY c.id_cliente, c.nome
ORDER BY total_gasto DESC;
```

RESULTADO

CLIENTE	TOTAL
Bruno	R$ 950
Ana	R$ 480
Carla	R$ 80
Diego	R$ 0

POR QUE COALESCE?

Converte NULL em zero para quem não fez compras.

Você já pensa em SQL!

A partir de agora, seu objetivo não é decorar comandos — é fazer perguntas aos dados.

MODELAGEM — Organizar antes de criar

CRUD — Manipular registros com cuidado

JOIN — Conectar informações

AGREGAÇÕES — Construir indicadores

EXIT TICKET · RESPONDA EM UMA FRASE

1. PK x FK? **2. UPDATE sem WHERE?** **3. WHERE x HAVING?**

SQL é uma forma de transformar dados em respostas.